

## ROTAS AÉREAS ENTRE CIDADES MARCIANAS

Depois da colonização e terraformação de Marte, em 2187 a Federação Marciana declarou-se independente da Terra e proporcionou aos seus habitantes uma malha aeroviária realizada por dirigíveis ligando os principais centros populacionais do planeta.

A Federação deseja fornecer aos seus usuários um aplicativo que permita verificar os caminhos entre essas principais cidades, através de viagens aéreas.



Para tanto, arquivo texto chamado CaminhosEntreCidadesMarte.txt contendo ids de cidades de origem e de destino, distância entre elas, tempo de percurso e preço da passagem é fornecido, com campos separados por “;”. Cada linha deste arquivo representa uma ligação direta entre uma cidade de origem e uma de destino.

Há também o arquivo texto chamado CidadesMarte.txt, contendo o id da cidade (chave de ordenação), seu nome e a coordenada cartesiana (x, y) proporcional às dimensões de um mapa de Marte, cujos campos também estão separados por “;”.

### Descrição dos arquivos

#### CidadesMarte.txt

IdCidade – inteiro  
NomeCidade – string  
CoordenadaX – real  
CoordenadaY – real

#### CaminhoEntreCidadesMarte.txt

idCidadeOrigem – inteiro  
idCidadeDestino – inteiro  
distancia – inteiro  
tempo – inteiro  
custo – inteiro

O aplicativo deverá encontrar e relacionar **todos os caminhos** existentes entre as cidades A e B indicadas pelo usuário, de acordo com a técnica de busca escolhida pelo usuário (backtracking com pilhas ou backtracking com recursão) e indicar também o caminho mais vantajoso, de acordo com o critério escolhido pelo usuário (menor distância, menor tempo, menor custo).

Os nomes dos arquivos de cidades e de caminhos devem ser solicitados ao usuário através de um OpenFileDialog especificando qual arquivo está sendo aberto. O arquivo de cidades deve ser armazenado em um vetor de objetos da classe Cidade. Nesse vetor se fará pesquisa binária pelo id da cidade, para que se tenha acesso às coordenadas dessa cidade, para que seja possível posicionar os extremos das linhas retas que determinam as ligações diretas entre cidades que fazem parte dos caminhos encontrados.

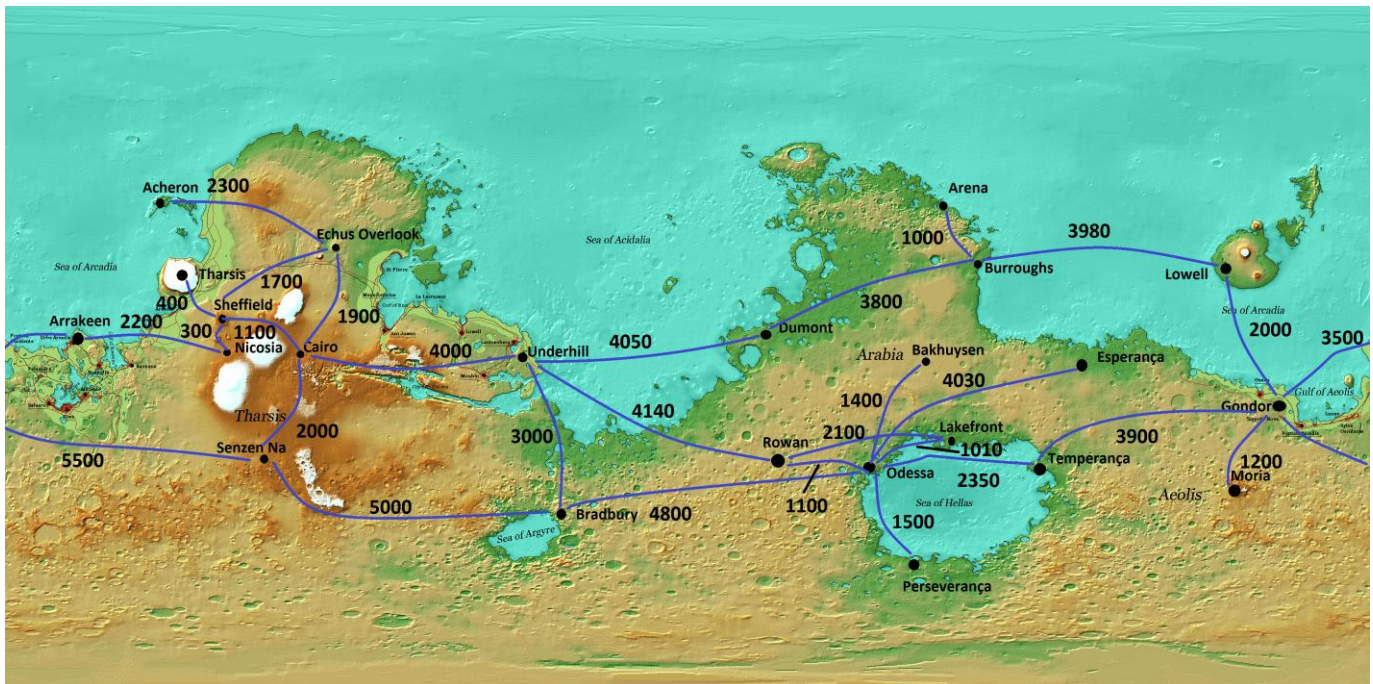
O arquivo de ligações diretas entre cidades deverá ser aberto, lido totalmente e armazenado em uma matriz de adjacências em que cada posição contém um objeto da classe Ligacao, que representa uma

ligação direta entre duas cidades e contém os ids das duas cidades diretamente ligadas, a distância, o tempo e o custo do percurso entre elas. As ligações diretas são bidirecionais.

A matriz será quadrada e sua ordem (número de linhas x número de colunas) será a quantidade de cidades armazenadas no vetor. Cada linha e cada coluna da matriz correspondem aos índices em que as cidades de origem e de destino estão armazenadas no vetor de cidades (posição da cidade de origem é o índice de linha e a posição da cidade de destino é o índice de coluna da matriz).

O id da cidade não corresponde a um índice, por isso é que precisamos buscar a cidade no vetor para saber seu índice, e este índice será o índice de linha e de coluna dessa cidade na matriz.

Conhecendo-se o id de uma cidade, para se descobrir seu nome deve-se usar a pesquisa binária no vetor para encontrar a cidade com esse id e, daí, acessar seu nome e coordenada. Conhecendo-se o nome de uma cidade desejada, para se conhecer seu id deverá se usar o nome como chave de busca sequencial.



Logo após a leitura e armazenamento das cidades no vetor de cidades, este deverá ser percorrido e os objetos de cada cidade devem ser armazenados nos dois comboboxes (cbxOrigem e cbxDestino), pois o usuário os usará para escolher as cidades cujos caminhos deseja buscar. Num ComboBox, podemos definir qual texto vai aparecer na tela e depois pegar o objeto de volta. Use a propriedade **DisplayMember** para mostrar o nome da cidade e a propriedade **SelectedItem** para recuperar o objeto completo que o usuário selecionou no combobox.

Como o mapa no seu tamanho original não caberá na tela deve-se continuar permitindo que ele se ajuste ao tamanho do painel de exibição. Isso mudará as coordenadas de exibição de cada cidade no mapa **proporcionalmente** à mudança da altura y e largura x do painel, numa proporção entre sua largura e sua com a coordenada (X, Y) original da cidade, já que o mapa poderá variar de tamanho caso você redimensione o formulário. Assim, como as coordenadas X e Y são proporcionais, para determinar o pixel (x, y) exato da cidade no mapa que está sendo exibido, a largura do painel deve ser multiplicada pela coordenada X da cidade e a altura do painel deve ser multiplicada pela coordenada Y da cidade. Por exemplo, se o mapa estiver sendo exibido com largura de 1200 e altura de 600 pixels, para a cidade de Acheron do exemplo acima as coordenadas onde o círculo deve ser exibido nesse mapa será (136, 173). Você pode escrever o nome da cidade abaixo ou acima do círculo.



### IMPORTANTE

- Desenvolver em C# no Visual Studio, em dupla;
- Comentar adequadamente o programa e o código programado;
- Nomear os identificadores de forma adequada;
- No início dos arquivos fonte, digitar comentário com os RAs e nomes dos alunos;
- Aproveitem as aulas da disciplina para o desenvolvimento e redação do relatório;
- **Relatório de desenvolvimento** deve ser feito num arquivo **PDF** cujo nome é: RA1\_RA2\_ED.PDF (exemplo: 25101\_25192\_ED.pdf). Deve conter imagens da execução da busca de caminhos;
- Entrega: **18/09/2026** pelo Google Classroom
- Material a ser entregue: pasta **do projeto completa, contendo também os arquivos de dados e o relatório em PDF compactados em um único arquivo, cujo nome será RA1\_RA2\_ED4.zip** (25101\_25192\_ED4.zip, por exemplo).

### Fontes de informação:

<https://www.deviantart.com/axiaterraartunion/art/Mars-Political-Map-294675891>

<http://www.teses.usp.br/teses/disponiveis/8/8147/tde-21102009-170528/pt-br.php>

[https://en.wikipedia.org/wiki/Mars\\_trilogy](https://en.wikipedia.org/wiki/Mars_trilogy)

